

Manual Técnico — GoLite

Este manual describe la estructura interna y el funcionamiento del código fuente del intérprete de GoLite.

1. Tecnologías

- **Java 21** — lenguaje de implementación del intérprete y la interfaz.
- **JFLEX** — generador del analizador léxico a partir de `lexer.flex`.
- **CUP** — generador del analizador sintáctico a partir de `parser.cup`.
- **Maven** — construcción del proyecto y manejo de dependencias.
- **Swing + RSyntaxTextArea** — interfaz gráfica y editor de código.

2. Arquitectura general

El intérprete procesa el código fuente en las siguientes fases:

1. **Análisis léxico:** el lexer (generado por JFLEX) lee el código y lo convierte en una secuencia de tokens. Los caracteres no válidos se registran como errores léxicos.
2. **Análisis sintáctico:** el parser (generado por CUP) valida la estructura gramatical de los tokens y, en el proceso, construye el Árbol de Sintaxis Abstracta (AST). Los errores de estructura se registran como errores sintácticos.
3. **Análisis semántico e interpretación:** se recorre el AST con el patrón Visitor. Durante el recorrido se valida la coherencia del programa (tipos, ámbitos, declaraciones) y se ejecutan las instrucciones. Las incoherencias se registran como errores semánticos.
4. **Salida:** el resultado de la ejecución se muestra en la consola, y los tokens y errores recolectados se muestran en sus respectivos reportes.

3. Estructura del código

El código está organizado en paquetes bajo `olc1.golite`:

- `Main` — punto de entrada; lanza la interfaz gráfica.
- `ast` — definición del Árbol de Sintaxis Abstracta.
 - `ast.exp` — nodos de expresión (operaciones aritméticas, comparaciones, lógicas, literales y funciones embebidas que devuelven valor).
 - `ast.stm` — nodos de sentencia (declaraciones, asignaciones, control de flujo, bloques, impresión).
- `symbols` — manejo de entornos y tabla de símbolos (`Environment`, `SymbolEntry`, `GoliteType`).
- `reports` — representación de errores y tokens (`GoliteError`, `SemanticException`, `Token`).
- `views` — interfaz gráfica (`GoliteFrame`, `GoliteMenuBar`, `MainPanel`, `EditorPanel`, `ReportFrame`).
- `visitor` — patrón Visitor.
 - `visitor.interpreter` — el intérprete (`InterpreterVisitor`).
 - `visitor.interpreter.value` — tipos de valor en tiempo de ejecución.
 - `visitor.interpreter.control` — señales internas de `break` y `continue`.

Las especificaciones del lexer y del parser están en `src/main/jflex/lexer.flex` y `src/main/cup/parser.cup`.

4. Componentes clave

Lexer (`lexer.flex`)

Define los patrones léxicos del lenguaje: números, identificadores, cadenas, runes, operadores, símbolos, palabras reservadas y comentarios. Cada token reconocido se entrega al parser y, al mismo tiempo, se almacena en una lista para el reporte de tokens. Los caracteres no reconocidos se acumulan en una lista de errores léxicos.

Parser (`parser.cup`)

Define la gramática del lenguaje y las reglas de precedencia de operadores. Por cada regla reconocida, las acciones semánticas construyen el nodo correspondiente del AST. El símbolo inicial produce una lista de instrucciones que representa el programa completo. Los errores de estructura se acumulan en una lista de errores sintácticos.

AST y patrón Visitor

Cada nodo del AST implementa la interfaz `ASTNode`, que define el método `accept`. Para evitar exponer los campos internos, cada nodo publica una clase interna `Context` que el Visitor recibe como parámetro. La interfaz `Visitor` declara un método `visit` por cada tipo de nodo, lo que obliga a manejar todos los casos del lenguaje.

Intérprete (`InterpreterVisitor`)

Implementa la interfaz `Visitor` y recorre el AST para ejecutar el programa. Por cada nodo evalúa la operación o ejecuta la sentencia correspondiente, validando los tipos involucrados. La salida del programa se va acumulando en una cadena de texto que luego se muestra en la consola.

Valores en tiempo de ejecución

Los valores se representan con la interfaz sellada `ValueWrapper` y sus implementaciones: `IntValue`, `DecimalValue`, `StringValue`, `BoolValue`, `RuneValue` y `VoidValue`. Cada valor conoce su tipo y su ubicación (línea y columna).

Entornos y tabla de símbolos

La clase `Environment` representa un ámbito y mantiene una referencia a su ámbito padre, formando una cadena de entornos. Esto permite el manejo correcto de variables globales y locales, y el ocultamiento de variables (shadowing) entre bloques. Cada bloque y cada ciclo `for` crean su propio entorno hijo, que se descarta al terminar.

Manejo de errores

Todos los errores (léxicos, sintácticos y semánticos) se representan con la clase `GoliteError`, que guarda el tipo, la descripción, la línea y la columna. Los errores semánticos se propagan mediante `SemanticException`. Las sentencias `break` y `continue` se implementan con señales internas (`BreakSignal`, `ContinueSignal`) que el ciclo `for` captura para controlar el flujo; si se usan fuera de un ciclo, se reportan como error.

5. Manejo de tipos

El lenguaje es de tipado estático: una vez declarada, una variable no puede cambiar de tipo. La única conversión implícita permitida es de `int` a `float64`. Las operaciones validan los tipos de sus operandos y reportan un error semántico ante combinaciones inválidas (por ejemplo,

comparar el orden de dos cadenas, aplicar operadores lógicos a valores no booleanos, o dividir entre cero).

6. Flujo al presionar "Ejecutar"

1. Se crea el lexer con el contenido del editor y se le pasa al parser.
2. El parser analiza la entrada y construye el AST.
3. Si hubo errores léxicos o sintácticos, no se interpreta y se avisa en la consola.
4. Si el análisis fue correcto, el `InterpreterVisitor` recorre el AST y ejecuta el programa.
5. La salida acumulada se muestra en la consola. Si ocurre un error semántico, se muestra su mensaje con la ubicación.